(

inicio( ) ( ) ( ) ( ) ( ) ( ) ( ) ( )

instrucciones( )

functions func saludar( )

) (block { }

( )

instrucciones( )

imprimir println( )

( )

expr "¡Hola, mundo!"

instrucciones( )

functions func obtenerNumero( ) ( )

) (valRet( )

type int

block { }

( )

instrucciones( )

retorno return( )

expr 42

instrucciones( )

functions func saludarPersona( ) ( )

nombre( )

type string

block { }

( )

instrucciones( )

imprimir println( )

( ) ,( )

expr "¡Hola," expr( ) +( )

expr nombre expr "!"

instrucciones( )

functions func sumar( ) ( ) ( )

a( ) , b( )

type int type int

valRet( )

type int

block { }

( )

instrucciones( )

retorno return( )

expr( ) +( )

expr a expr b

instrucciones( )

functions func factorial( ) ( ) ( )

n( )

type int

valRet( )

type int

block { }

( ) ( )

instrucciones( )

sIf if( ) ( )

expr( ) <=( )

expr n expr 1

block { }

( )

instrucciones( )

retorno return( )

expr 1

instrucciones( )

retorno return( )

expr( ) *( )

expr n expr( )

varCallStatement factorial( )

( )

expr( ) -( )

expr n expr 1

instrucciones( )

functions func ackermann( ) ( ) ( )

m( ) , n( )

type int type int

valRet( )

type int

block { }

( )

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr m expr 0

block { }

( )

instrucciones( )

retorno return( )

expr( ) +( )

expr n expr 1

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) >( )

expr m expr 0

expr( ) ==( )

expr n expr 0

block { }

( )

instrucciones( )

retorno return( )

expr( )

varCallStatement ackermann( )

( ) ,( )

expr( ) -( )

expr m expr 1

expr 1

block { }

( )

instrucciones( )

retorno return( )

expr( )

varCallStatement ackermann( )

( ) ,( )

expr( ) -( )

expr m expr 1

expr( )

varCallStatement ackermann( )

( ) ,( )

expr m expr( ) -( )

expr n expr 1

instrucciones( )

functions func hanoi( ) ( )

n( ) , origen( ) , auxiliar( ) , destino( )

type int type string type string type string

block { }

( ) ( ) ( ) ( )

instrucciones( )

sIf if( ) ( )

expr( ) ==( )

expr n expr 1

block { }

( ) ( )

instrucciones( )

imprimir println( )

( ) ,( ) ,( ) ,( )

expr "Mover disco 1 de" expr origen expr "a" expr destino

instrucciones( )

retorno return

instrucciones( )

varCallStatement hanoi( )

( ) ,( ) ,( ) ,( )

expr( ) -( )

expr n expr 1

expr origen expr destino expr auxiliar

instrucciones( )

imprimir println( )

( ) ,( ) ,( ) ,( ) ,( ) ,( )

expr "Mover disco" expr n expr "de" expr origen expr "a" expr destino

instrucciones( )

varCallStatement hanoi( )

( ) ,( ) ,( ) ,( )

expr( ) -( )

expr n expr 1

expr auxiliar expr origen expr destino

instrucciones( )

functions func main( )

) (block { }

( ) ( )

instrucciones( )

varDcl puntos( ) ( )

assign := expr 0

instrucciones( )

imprimir println( )

( )

expr "=== Archivo de prueba de funciones ==="

instrucciones( )

imprimir println( )

( )

expr "==== Funciones no recursivas sin parámetros ===="

instrucciones( )

varDcl puntosFuncionesSimples( ) ( )

assign := expr 0

instrucciones( )

imprimir println( )

( )

expr "Función sin retorno:"

instrucciones( )

imprimir println( )

( )

expr "\n\n###Validacion Manual"

instrucciones( )

varCallStatement saludar(

)

instrucciones( )

imprimir println( )

( )

expr "\nFunción con retorno:"

instrucciones( )

varDcl numero( ) ( )

assign := expr( )

varCallStatement obtenerNumero(

)

instrucciones( )

imprimir println( )

( ) ,( )

expr "El número obtenido es:" expr numero

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr numero expr 42

block { }

( ) ( )

instrucciones( )

varAsign puntosFuncionesSimples =( )

expr( ) +( )

expr puntosFuncionesSimples expr 5

instrucciones( )

imprimir println( )

( )

expr "OK Funciones no recursivas sin parámetros: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Funciones no recursivas sin parámetros: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== Funciones no recursivas con parámetros ===="

instrucciones( )

varDcl puntosFuncionesParametros( ) ( )

assign := expr 0

instrucciones( )

imprimir println( )

( )

expr "Función con parámetros sin retorno:"

instrucciones( )

imprimir println( )

( )

expr "\n\n###Validacion Manual"

instrucciones( )

varCallStatement saludarPersona( )

( )

expr "Juan"

instrucciones( )

imprimir println( )

( )

expr "\nFunción con parámetros y retorno:"

instrucciones( )

varDcl resultadoSuma( ) ( )

assign := expr( )

varCallStatement sumar( )

( ) ,( )

expr 10 expr 20

instrucciones( )

imprimir println( )

( ) ,( )

expr "La suma de 10 y 20 es:" expr resultadoSuma

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr resultadoSuma expr 30

block { }

( ) ( )

instrucciones( )

varAsign puntosFuncionesParametros =( )

expr( ) +( )

expr puntosFuncionesParametros expr 5

instrucciones( )

imprimir println( )

( )

expr "OK Función con parámetros y retorno: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Función con parámetros y retorno: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== Funciones recursivas ===="

instrucciones( )

varDcl puntosFuncionesRecursivas( ) ( )

assign := expr 0

instrucciones( )

imprimir println( )

( )

expr "Factorial:"

instrucciones( )

varDcl resultadoFactorial( ) ( )

assign := expr( )

varCallStatement factorial( )

( )

expr 5

instrucciones( )

imprimir println( )

( ) ,( )

expr "Factorial de 5:" expr resultadoFactorial

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr resultadoFactorial expr 120

block { }

( ) ( )

instrucciones( )

varAsign puntosFuncionesRecursivas =( )

expr( ) +( )

expr puntosFuncionesRecursivas expr 3

instrucciones( )

imprimir println( )

( )

expr "OK Función factorial: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Función factorial: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nTorres de Hanoi:"

instrucciones( )

imprimir println( )

( )

expr "\n\n###Validacion Manual"

instrucciones( )

varCallStatement hanoi( )

( ) ,( ) ,( ) ,( )

expr 3 expr "A" expr "B" expr "C"

instrucciones( )

varAsign puntosFuncionesRecursivas =( )

expr( ) +( )

expr puntosFuncionesRecursivas expr 3

instrucciones( )

imprimir println( )

( )

expr "\nAckermann:"

instrucciones( )

varDcl resultadoAckermann( ) ( )

assign := expr( )

varCallStatement ackermann( )

( ) ,( )

expr 3 expr 2

instrucciones( )

imprimir println( )

( ) ,( )

expr "Ackermann de (3, 2):" expr resultadoAckermann

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr resultadoAckermann expr 29

block { }

( ) ( )

instrucciones( )

varAsign puntosFuncionesRecursivas =( )

expr( ) +( )

expr puntosFuncionesRecursivas expr 4

instrucciones( )

imprimir println( )

( )

expr "OK Función ackermann: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Función ackermann: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== Atoi ===="

instrucciones( )

varDcl puntosAtoi( ) ( )

assign := expr 0

instrucciones( )

imprimir println( )

( )

expr "Conversión de string a int:"

instrucciones( )

varDcl numeroEntero( ) ( )

assign := expr Atoi( )

( )

expr "123"

instrucciones( )

imprimir println( )

( ) ,( )

expr "\"123\" convertido a int:" expr numeroEntero

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr numeroEntero expr 123

block { }

( ) ( )

instrucciones( )

varAsign puntosAtoi =( )

expr( ) +( )

expr puntosAtoi expr 2

instrucciones( )

imprimir println( )

( )

expr "OK Atoi: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Atoi: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== parseFloat ===="

instrucciones( )

varDcl puntosParseFloat( ) ( )

assign := expr 0

instrucciones( )

imprimir println( )

( )

expr "Conversión de string a float64:"

instrucciones( )

varDcl numeroDecimal1( ) ( )

assign := expr parseFloat( )

( )

expr "123.45"

instrucciones( )

imprimir println( )

( ) ,( )

expr "\"123.45\" convertido a float64:" expr numeroDecimal1

instrucciones( )

varDcl numeroDecimal2( ) ( )

assign := expr parseFloat( )

( )

expr "123"

instrucciones( )

imprimir println( )

( ) ,( )

expr "\"123\" convertido a float64:" expr numeroDecimal2

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) ==( )

expr numeroDecimal1 expr 123.45

expr( ) ==( )

expr numeroDecimal2 expr 123.0

block { }

( ) ( )

instrucciones( )

varAsign puntosParseFloat =( )

expr( ) +( )

expr puntosParseFloat expr 2

instrucciones( )

imprimir println( )

( )

expr "OK parseFloat: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X parseFloat: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== typeOf ===="

instrucciones( )

varDcl puntosTypeOf( ) ( )

assign := expr 0

instrucciones( )

imprimir println( )

( )

expr "Obtener tipo de datos:"

instrucciones( )

varDcl entero( ) ( )

assign := expr 42

instrucciones( )

varDcl tipoEntero( ) ( )

assign := expr typeOf( )

( )

expr entero

instrucciones( )

imprimir println( )

( ) ,( )

expr "Tipo de 42:" expr tipoEntero

instrucciones( )

varDcl decimal( ) ( )

assign := expr 3.14159

instrucciones( )

varDcl tipoDecimal( ) ( )

assign := expr typeOf( )

( )

expr decimal

instrucciones( )

imprimir println( )

( ) ,( )

expr "Tipo de 3.14159:" expr tipoDecimal

instrucciones( )

varDcl texto( ) ( )

assign := expr "Hola, mundo!"

instrucciones( )

varDcl tipoTexto( ) ( )

assign := expr typeOf( )

( )

expr texto

instrucciones( )

imprimir println( )

( ) ,( )

expr "Tipo de \"Hola, mundo!\":" expr tipoTexto

instrucciones( )

varDcl booleano( ) ( )

assign := expr true

instrucciones( )

varDcl tipoBooleano( ) ( )

assign := expr typeOf( )

( )

expr booleano

instrucciones( )

imprimir println( )

( ) ,( )

expr "Tipo de true:" expr tipoBooleano

instrucciones( )

varDclSlice slice( ) ( ) ( ) { }

assign := nuevoSlice [] type int ( )

contenidoSlice( ) ,( ) ,( )

expr 1 expr 2 expr 3

instrucciones( )

varDcl tipoSlice( ) ( )

assign := expr typeOf( )

( )

expr slice

instrucciones( )

imprimir println( )

( ) ,( )

expr "Tipo de []int{1, 2, 3}:" expr tipoSlice

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) ==( )

expr tipoEntero expr "int"

expr( ) ==( )

expr tipoDecimal expr "float64"

expr( ) ==( )

expr tipoTexto expr "string"

expr( ) ==( )

expr tipoBooleano expr "bool"

expr( ) ==( )

expr tipoSlice expr "[]int"

block { }

( ) ( )

instrucciones( )

varAsign puntosTypeOf =( )

expr( ) +( )

expr puntosTypeOf expr 1

instrucciones( )

imprimir println( )

( )

expr "OK typeOf: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X typeOf: incorrecto"

instrucciones( )

varAsign puntos =( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr puntosFuncionesSimples expr puntosFuncionesParametros

expr puntosFuncionesRecursivas

expr puntosAtoi

expr puntosParseFloat

expr puntosTypeOf

instrucciones( )

imprimir println( )

( )

expr "\n=== Tabla de Resultados ==="

instrucciones( )

imprimir println( )

( )

expr "+----------------------------------+--------+-------+"

instrucciones( )

imprimir println( )

( )

expr "| Característica | Puntos | Total |"

instrucciones( )

imprimir println( )

( )

expr "+----------------------------------+--------+-------+"

instrucciones( )

imprimir println( )

( ) ,( ) ,( )

expr "| Funciones sin parámetros | " expr puntosFuncionesSimples expr " | 5 |"

instrucciones( )

imprimir println( )

( ) ,( ) ,( )

expr "| Funciones con parámetros | " expr puntosFuncionesParametros expr " | 5 |"

instrucciones( )

imprimir println( )

( ) ,( ) ,( )

expr "| Funciones recursivas | " expr puntosFuncionesRecursivas expr " | 10 |"

instrucciones( )

imprimir println( )

( ) ,( ) ,( )

expr "| Atoi | " expr puntosAtoi expr " | 2 |"

instrucciones( )

imprimir println( )

( ) ,( ) ,( )

expr "| parseFloat | " expr puntosParseFloat expr " | 2 |"

instrucciones( )

imprimir println( )

( ) ,( ) ,( )

expr "| typeOf | " expr puntosTypeOf expr " | 1 |"

instrucciones( )

imprimir println( )

( )

expr "+----------------------------------+--------+-------+"

instrucciones( )

imprimir println( )

( ) ,( ) ,( )

expr "| TOTAL | " expr puntos expr " | 25 |"

instrucciones( )

imprimir println( )

( )

expr "+----------------------------------+--------+-------+"

)